System Architecture & Deployment Topology/Arrangement

The AEGIS system uses a Distributed Cloud Architecture by exploiting production-ready free-tier platforms. This architecture isolates the presentation, and application orchestration layers as well as the data persistence, and untrusted code execution code execution sandbox.

Component Selection & Comparative Analysis

- Frontend Hosting: Vercel
    - **Approach**: Serverless deployment by using Vercels's Edge Network.
    - **Alternatives Considered:** Self-hosting on a traditional Linux server, AWS S3 and CloudFront.
        - **Justification vs. Alternatives:** AWS S3 and CloudFront offer enterprise scaling, its configuration carries variable data-egress financial risks. Vercel serves static web assets through a global Content Delivery Network (CDN) straight out of the box without any cost. This guarantees fast initial page loads globally.
    - **Non Functional Requirement Alignment: Scalability & Concurrency:** The CDN design can handle traffic spikes up to and beyond 500 concurrent users without UI latency.
        - **Maintainability:** Inherit GitHub integration triggers automatic compilation and deployment upon code merges, meeting the required **2-hour deployment window requirement.**
- Backend Orchestration Layer: FastAPI on Koyeb
    - **Approach:** Containerised deployment on Koyeb's Free Tier (Micro Instance).
    - **Alternatives Considered:** Render(Free Tier), Railway (Paid).
        - **Justification vs. Alternatives:** Traditional free tiers (like Render) put backend applications in a "sleep" state after 15 minutes of inactivity, which causes a 30-to-50 second "cold start" delay. Koyeb's free tier provides an instance that remains active continuously, which eliminates the cold starts.
    - **Non Functional Requirement Alignment:**
        - **Performance:** Keeping the backend instance awake ensures that API routing begins processing immediately which is required for achieving a **< 2-second response time.**
- Data Persistence Layer: Supabase (Managed PostgreSQL)
    - **Approach:** Cloud-managed PostgreSQL instance by Supabase.
    - **Alternatives Considered:** Self-hosted PostgreSQL via Docker, SQLite.
        - **Justification vs. Alternatives:** SQLite does not have the required concurrent write capabilities. Self-hosting PostgreSql on free-tier heavily consumes limited memory resources, risking Out-Of-Memory (OOM) crashes. Supabase provides an isolated database environment at no cost.
    - **Non Functional Requirement Alignment:**

    - **Security:** Natively, Supabase manages **AES-256 encryption at rest and SSL encryption in transit**, satisfying data privacy requirements.
    - **Scalability:** Includes Supavisor for database connection pooling, preventing connection exhaustion when 500 concurrent users execute database queries.
-